

Guía de Aprendizaje por Refuerzo

Del concepto de agente al Deep Q-Network (DQN)

Con ejemplos en Python usando Gymnasium (OpenAI Gym) y PyTorch

Generado como material de estudio

1. Idea general

En **Aprendizaje por Refuerzo** (Reinforcement Learning, RL) queremos que un **agente** aprenda, por prueba y error, a moverse dentro de un **entorno** hasta descubrir cómo llegar a una meta. El agente no recibe instrucciones explícitas de qué hacer: en cambio, el entorno le entrega una **recompensa** (positiva o negativa) después de cada acción, y el agente ajusta su comportamiento para maximizar la suma de recompensas que obtiene a lo largo del tiempo.

Todo el problema se puede resumir en tres piezas fundamentales que se repiten una y otra vez:

Estado	La "foto" de la situación actual del agente dentro del entorno (por ejemplo, en qué casilla del mapa está).
Acción	Lo que el agente puede hacer en ese estado (por ejemplo: moverse arriba, abajo, izquierda o derecha).
Función de recompensa	La regla que castiga o premia al agente según el resultado de su acción (por ejemplo: +10 por llegar a la meta, -1 por caer en un hoyo).

*Ciclo básico: **Estado** → **Agente** → **Acción** → (el entorno reacciona y entrega un nuevo Estado + una Recompensa) → y el ciclo se repite.*

2. Las piezas del sistema

Para que el ciclo anterior funcione, necesitamos definir con claridad cada una de las piezas que participan en la interacción entre el agente y su mundo:

Entorno	El "mundo" simulado donde vive el agente (el mapa, el juego, la simulación). Es quien decide las reglas.
Agente	El programa que toma decisiones; es quien aprende.
Estado	La situación actual, tal como la percibe el agente.
Acción	La decisión que toma el agente en un estado dado.
Recompensa + nuevo estado	Lo que el entorno le devuelve al agente inmediatamente después de ejecutar una acción.

El ciclo completo

El flujo de información se puede describir así: el **Agente** observa el **Estado** actual y elige una **Acción**. Esa acción se ejecuta dentro del **Entorno**, el cual responde entregando una **Recompensa** y un **nuevo Estado**. Con esa información, el agente vuelve a decidir su siguiente acción, y así sucesivamente hasta terminar el episodio (por ejemplo, al llegar a la meta o caer en un hoyo).

Entorno → entrega Recompensa y nuevo Estado → el Agente vuelve a decidir.

3. El modelo: ¿qué tan conveniente es cada acción?

El **modelo** es la parte del agente que estima qué tan conveniente es tomar cada acción posible en un estado determinado. Esta estimación se representa mediante un **valor Q** (de "Quality", calidad): mientras más alto es el valor Q de una acción, mejor se espera que sea tomarla.

Q	Es el valor numérico estimado para cada par (estado, acción): qué tan buena es esa acción en ese estado.
Deep Q-Network (DQN)	Una red neuronal profunda que aproxima los valores Q, en lugar de guardarlos en una tabla.
Network (red)	Los valores Q son aproximados por una red neuronal cuando hay demasiados estados para usar una tabla.

En problemas pequeños (como un mapa de 4x4 casillas) se puede usar una **tabla Q** que guarda un valor por cada combinación de estado y acción. Pero en problemas grandes o continuos (como imágenes de un videojuego), la tabla sería enorme o imposible de construir. Ahí es donde entra el **Deep Q-Network**: en vez de una tabla, usamos una red neuronal que recibe el estado como entrada y devuelve, como salida, el valor Q estimado para cada acción posible.

4. Explorar vs. explotar

Un agente que solo hace lo que ya sabe que funciona nunca descubre caminos mejores. Y un agente que solo prueba cosas al azar nunca aprovecha lo que ya aprendió. Por eso se balancean dos estrategias:

Explorar	Elegir una acción al azar, sin usar lo aprendido, para descubrir nuevas posibilidades.
Explotar	Elegir la acción con el mayor valor Q conocido hasta el momento, usando lo ya aprendido.
Epsilon (ϵ)	Probabilidad de explorar en lugar de explotar. Epsilon = 1 significa "explorar siempre" (100% acciones al azar); epsilon = 0 significa "explotar siempre".

Una estrategia muy común es **epsilon-greedy**: al principio del entrenamiento, epsilon empieza alto (cercano a 1) para que el agente explore mucho el entorno, ya que aún no sabe nada. A medida que el agente entrena y su tabla o red Q mejora, epsilon va bajando poco a poco (decaimiento de epsilon), de modo que el agente empieza a confiar más en lo aprendido y explota más.

5. La función de recompensa y el Q objetivo

Cada vez que el agente ejecuta una acción, el entorno le entrega una **recompensa** inmediata. Pero el agente no solo quiere maximizar la recompensa inmediata: también le importa el valor de las oportunidades futuras que esa acción le abre. Por eso, cuando actualizamos el valor Q de una acción, no usamos solo la recompensa recibida, sino un **Q objetivo** que combina la recompensa inmediata con una

estimación del futuro.

$$Q_{\text{objetivo}} = \text{Recompensa} + \gamma \times (\text{mejor } Q \text{ del nuevo estado})$$

Donde γ (gamma) es el **factor de descuento**, un número entre 0 y 1 que indica cuánto le importan al agente las recompensas futuras frente a la inmediata. Un gamma cercano a 1 hace que el agente valore mucho el futuro; un gamma cercano a 0 lo hace "cortoplacista", enfocado solo en la recompensa inmediata.

Ejemplo numérico

Supongamos que el agente recibe una recompensa de **-1** por el paso que acaba de dar, usa un factor de descuento $\gamma = 0.9$, y el mejor valor Q estimado para las acciones del nuevo estado es **22.1**. El cálculo del Q objetivo sería:

```
Q_objetivo = recompensa + gamma * mejor_Q_nuevo_estado
Q_objetivo = -1 + 0.9 * 22.1
Q_objetivo = -1 + 19.89
Q_objetivo = 18.89
```

Este valor de **18.89** se compara luego con el valor Q que la tabla (o la red neuronal) tenía estimado para esa acción en ese estado. La diferencia entre ambos (llamada *error temporal* o *TD-error*) es lo que se usa para corregir la estimación: si el Q objetivo es mayor que el estimado, la acción era mejor de lo que se pensaba y su valor Q sube un poco; si es menor, el valor Q baja un poco.

*En resumen: el agente compara "lo que esperaba" (Q estimado) contra "lo que en realidad conviene" (Q objetivo) y ajusta su conocimiento en esa dirección. Esta es exactamente la **ecuación de Bellman** aplicada a Q-learning.*

6. Código completo: Q-Learning en Gymnasium

A continuación, el código completo de un agente que aprende a resolver el entorno **FrozenLake-v1** (una cuadrícula donde debe llegar a la meta sin caer en un hoyo) usando **Q-learning** con una tabla Q. Más abajo se explica cada línea en detalle.

```

import numpy as np
import gymnasium as gym

# Crear el entorno: cuadrícula 4x4, sin resbalones (determinista)
env = gym.make("FrozenLake-v1", is_slippery=False)

n_estados = env.observation_space.n
n_acciones = env.action_space.n

# Tabla Q: una fila por estado, una columna por accion, todo en cero
Q = np.zeros((n_estados, n_acciones))

# Hiperparametros del entrenamiento
alpha = 0.8          # tasa de aprendizaje
gamma = 0.95         # factor de descuento (importancia del futuro)
epsilon = 1.0         # probabilidad inicial de explorar (100%)
epsilon_min = 0.01    # piso minimo de exploracion
epsilon_decay = 0.995 # que tan rapido baja epsilon
episodios = 2000      # cuantas partidas completas se juegan
max_pasos = 100       # limite de pasos por partida

recompensas_por_episodio = []

for episodio in range(episodios):
    estado, _ = env.reset()
    recompensa_total = 0

    for paso in range(max_pasos):
        if np.random.rand() < epsilon:
            accion = env.action_space.sample()      # EXPLORAR
        else:
            accion = np.argmax(Q[estado])           # EXPLOTAR

        nuevo_estado, recompensa, terminado, truncado, _ = env.step(accion)

        mejor_q_siguiente = np.max(Q[nuevo_estado])
        Q[estado, accion] = Q[estado, accion] + alpha * (
            recompensa + gamma * mejor_q_siguiente - Q[estado, accion]
        )

        estado = nuevo_estado
        recompensa_total += recompensa

        if terminado or truncado:
            break

    epsilon = max(epsilon_min, epsilon * epsilon_decay)
    recompensas_por_episodio.append(recompensa_total)

env.close()
print("Tabla Q final:")
print(Q)

```

Explicación línea por línea

Código	Explicación
<code>import numpy as np</code>	Importa NumPy, usado para crear y operar la tabla Q como una matriz numérica.
<code>import gymnasium as gym</code>	Importa Gymnasium, la librería que provee entornos de RL ya implementados.
<code>env = gym.make(...)</code>	Crea el entorno FrozenLake. <i>is_slippery=False</i> hace que el piso no sea resbaladizo, así el movimiento es predecible.
<code>n_estados = env.observation_space.n</code>	Obtiene cuántos estados distintos existen (una por cada casilla del mapa).
<code>n_acciones = env.action_space.n</code>	Obtiene cuántas acciones puede tomar el agente (arriba, abajo, izquierda, derecha).
<code>Q = np.zeros((n_estados, n_acciones))</code>	Crea la tabla Q, inicialmente en cero: aún no se sabe nada sobre qué acción conviene en cada estado.
<code>alpha = 0.8</code>	Tasa de aprendizaje: qué tan grande es cada corrección que se aplica a la tabla Q.
<code>gamma = 0.95</code>	Factor de descuento: qué tanto peso se le da a las recompensas futuras.
<code>epsilon = 1.0</code>	Al inicio del entrenamiento, el agente explora el 100% de las veces (no sabe nada aún).
<code>epsilon_decay = 0.995</code>	En cada episodio, epsilon se multiplica por este número para ir reduciendo la exploración.
<code>for episodio in range(episodios):</code>	Bucle principal: se juegan 2000 partidas completas (episodios) de entrenamiento.
<code>estado, _ = env.reset()</code>	Reinicia el entorno al comienzo de cada episodio y obtiene el estado inicial.
<code>for paso in range(max_pasos):</code>	Bucle interno: cada episodio dura como máximo 100 pasos.
<code>if np.random.rand() < epsilon: ... env.action_space.sample()</code>	Con probabilidad epsilon, se elige una acción completamente al azar (explorar).
<code>else: accion = np.argmax(Q[estado])</code>	Si no toca explorar, se elige la acción con el mayor valor Q conocido para el estado actual (explotar).

<code>nuevo_estado, recompensa, terminado, truncado, _ = env.step(accion)</code>	Ejecuta la acción en el entorno y recibe: el nuevo estado, la recompensa, y si el episodio terminó.
<code>mejor_q_siguiente = np.max(Q[nuevo_estado])</code>	Busca el mejor valor Q posible desde el nuevo estado; es la pieza "futuro" del Q objetivo.
<code>Q[estado, accion] = Q[estado, accion] + alpha * (recompensa + gamma * mejor_q_siguiente - Q[estado, accion])</code>	Ecuación de Bellman: actualiza el valor Q combinando el valor anterior con la diferencia entre el Q objetivo y el Q estimado, escalada por alpha.
<code>estado = nuevo_estado</code>	El nuevo estado pasa a ser el estado actual para el siguiente paso del bucle.
<code>if terminado or truncado: break</code>	Si el agente llegó a la meta, cayó en un hoyo, o se acabaron los pasos, termina el episodio.
<code>epsilon = max(epsilon_min, epsilon * epsilon_decay)</code>	Reduce epsilon gradualmente, sin bajar nunca del mínimo definido, para no dejar de explorar del todo.
<code>recompensas_por_episodio.append(recompensa_total)</code>	Guarda el resultado del episodio para poder analizar el progreso del agente más adelante.

7. De la tabla Q a una red neuronal (DQN)

Cuando el número de estados es muy grande (o infinito, como en imágenes o sensores continuos), ya no es posible guardar una tabla Q completa. La solución es reemplazar la tabla por una **red neuronal** que recibe el estado como entrada y predice el valor Q de cada acción posible como salida. A esta red se le llama **Deep Q-Network (DQN)**.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class DQN(nn.Module):
    def __init__(self, n_estados, n_acciones):
        super(DQN, self).__init__()
        self.fc1 = nn.Linear(n_estados, 128) # capa oculta 1
        self.fc2 = nn.Linear(128, 128) # capa oculta 2
        self.fc3 = nn.Linear(128, n_acciones) # capa de salida: un valor Q por accion

    def forward(self, x):
        x = F.relu(self.fc1(x)) # activacion ReLU tras la capa 1
        x = F.relu(self.fc2(x)) # activacion ReLU tras la capa 2
        return self.fc3(x) # salida sin activacion: son los valores Q
```

Explicación línea por línea

Código	Explicación
<code>import torch / torch.nn / torch.nn.functional</code>	Importa PyTorch y sus módulos para construir y entrenar redes neuronales.
<code>class DQN(nn.Module):</code>	Define la red neuronal como una clase que hereda de <code>nn.Module</code> , la base de todo modelo en PyTorch.
<code>self.fc1 = nn.Linear(n_estados, 128)</code>	Primera capa totalmente conectada: recibe el estado (tamaño <code>n_estados</code>) y produce 128 valores internos.
<code>self.fc2 = nn.Linear(128, 128)</code>	Segunda capa oculta: procesa aún más la información antes de llegar a la salida.
<code>self.fc3 = nn.Linear(128, n_acciones)</code>	Capa de salida: produce un valor Q por cada acción posible.
<code>def forward(self, x):</code>	Define cómo fluyen los datos a través de la red cuando se le pasa un estado <code>x</code> .
<code>x = F.relu(self.fc1(x))</code>	Aplica la capa 1 y luego la activación ReLU (deja pasar los valores positivos, convierte los negativos en 0).
<code>x = F.relu(self.fc2(x))</code>	Aplica la capa 2 y otra activación ReLU, permitiendo que la red aprenda relaciones no lineales.

```
return self.fc3(x)
```

Devuelve los valores Q finales, uno por acción, sin activación (son números reales, no probabilidades).

En un entrenamiento DQN completo, esta red se usa para estimar el Q de cada acción, se compara con el Q objetivo (recompensa + $\gamma \times$ mejor Q del siguiente estado, igual que en la sección 5) y se ajustan los pesos de la red mediante backpropagation para que sus predicciones se acerquen cada vez más al Q objetivo.

8. Resumen de conceptos clave

Entorno	El mundo donde actúa el agente y que define sus reglas.
Agente	Quien decide qué acción tomar y aprende con la experiencia.
Estado	La situación actual observada por el agente.
Acción	Lo que el agente decide hacer en un estado.
Recompensa	Señal numérica que indica qué tan buena o mala fue una acción.
Valor Q	Estimación de qué tan conveniente es una acción en un estado dado.
Tabla Q / DQN	Forma de almacenar (tabla) o aproximar (red neuronal) los valores Q.
Explorar / Explotar	Probar algo nuevo al azar vs. usar lo que ya se sabe que funciona.
Epsilon	Probabilidad de explorar; empieza alto y disminuye con el entrenamiento.
Gamma	Factor de descuento: importancia de las recompensas futuras.
Q objetivo	Recompensa inmediata + gamma $\times$ mejor Q del nuevo estado; guía la actualización de la tabla o red.